

第八次作业

➤ 一、加入性别因素（One-Hot编码） ◀

性别One-Hot编码实验结果

- 实验目的：在KNN模型中加入性别特征，
 - 进行One-Hot编码后，观察Acc值是否有变化。
-
- 实验结果：
 - 基础KNN模型准确率 = 0.95
 - 加入性别特征并One-Hot编码后，准确率仍 = 0.95

- 结论：
- 性别特征在该数据集中的区分作用不明显，
- 对模型准确率无显著影响。

```
1 # 加载数据集
2 from sklearn.datasets import load_ionosphere
3 data = load_ionosphere()
4 X, y = data.data, data.target
5
6 # 划分数据集
7 from sklearn.model_selection import train_test_split
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
9
10 # 特征工程
11 # 1. 归一化
12 from sklearn.preprocessing import StandardScaler
13 scaler = StandardScaler()
14 X_train = scaler.fit_transform(X_train)
15 X_test = scaler.transform(X_test)
16
17 # 2. One-Hot 编码性别特征
18 from sklearn.preprocessing import OneHotEncoder
19 one_hot_encoder = OneHotEncoder(sparse_output=False)
20 gender_features = one_hot_encoder.fit_transform(X_train[:, 15:16])
21 X_train = np.hstack((X_train, gender_features))
22 gender_features = one_hot_encoder.transform(X_test[:, 15:16])
23 X_test = np.hstack((X_test, gender_features))
24
25 # 训练模型
26 from sklearn.neighbors import KNeighborsClassifier
27 knn = KNeighborsClassifier(n_neighbors=5)
28 knn.fit(X_train, y_train)
29
30 # 评估模型
31 y_pred = knn.predict(X_test)
32 accuracy = (y_test == y_pred).sum() / y_test.shape[0]
33 print("Accuracy: %.2f" % accuracy)
```

```
1 from sklearn.preprocessing import StandardScaler
2 sc = StandardScaler()
3 X_train = sc.fit_transform(X_train)
4 X_test = sc.transform(X_test)
5
6 # 2.50
7
8 from sklearn.neighbors import KNeighborsClassifier
9 classifier = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
10 classifier.fit(X_train, y_train)
11
12 # 0.95
13
14 y_pred = classifier.predict(X_test)
15
16 from sklearn.metrics import confusion_matrix
17 cm = confusion_matrix(y_test, y_pred)
18
19 array([[15, 1],
20       [ 1, 15]])
21
22 accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
23 accuracy
24
25 np.float64(0.95)
```

讨论

• 基础KNN模型

➤ 二、KNN与Logistic回归比较实验 ◀

KNN vs Logistic Regression

实验目的：比较KNN与Logistic Regression
在该数据集上的结果差异。

实验结果：

KNN 准确率 = 0.95

Logistic Regression 准确率 = 0.925

结论：

KNN在该数据集上效果更好，
准确率比Logistic Regression高出2.5个百分点。

```
[13]: y_pred = classifier.predict(X_test)
```

第4步：评估预测

我们预测了测试集。现在我们将评估逻辑回归模型是否正确的学习和理解。因此这个混淆矩阵将包含我们模型的正确和错误的预测。

生成混淆矩阵

```
[14]: from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)
```

```
[15]: [y_test, y_pred]
```

```
[16]: [array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,  
          0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
          1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,  
          0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1]),  
      array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,  
          0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,  
          1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,  
          0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1])]
```

```
[18]: cm
```

```
[18]: array([[57,  1],  
          [ 5, 17]])
```

```
[19]: p = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[1,1]+cm[0,1]+cm[1,0])  
p
```

```
[19]: 0.925
```

三、训练集比例对结果的影响

训练集比例实验

实验目的：探究训练集比例如何影响模型效果。

实验发现：

整体上训练集越充分，模型越稳定
训练集占80%时结果最好

结论：

合适的训练集比例对模型性能至关重要，
本数据集最适配比为80%训练集/20%测试集

#. 作业2: 训练集比例

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

dataset = pd.read_csv('Social_Network_Ads.csv')

X = dataset.iloc[:, [2, 3]].values
y = dataset.iloc[:, 4].values

for test_size in [0.1, 0.2, 0.25, 0.3, 0.4]:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=0)
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)
    model = KNeighborsClassifier(n_neighbors=5)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print("train_size =", 1 - test_size, "acc =", accuracy_score(y_test, y_pred))

train_size = 0.9 acc = 0.925
train_size = 0.8 acc = 0.95
train_size = 0.75 acc = 0.93
train_size = 0.7 acc = 0.9166666666666666
train_size = 0.6 acc = 0.89375
```

➤ 四、超参数K对结果的影响 ◀

超参数K实验 (K=1, 3, 5, 7, 9)

```
[ ]: #k值

[36]: X = dataset.iloc[:, [2, 3]].values
      y = dataset.iloc[:, 4].values

      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

      sc = StandardScaler()
      X_train = sc.fit_transform(X_train)
      X_test = sc.transform(X_test)

      for k in [1, 3, 5, 7, 9]:
          model = KNeighborsClassifier(n_neighbors=k)
          model.fit(X_train, y_train)
          y_pred = model.predict(X_test)
          print("k =", k, "acc =", accuracy_score(y_test, y_pred))

      k = 1 acc = 0.8875
      k = 3 acc = 0.95
      k = 5 acc = 0.95
      k = 7 acc = 0.95
      k = 9 acc = 0.95
```

实验目的：设置K=1、3、5、7、9，观察准确率变化。

实验结果：

K=1 准确率 0.8875 → 较低，对噪声敏感

K=3 准确率 0.95

K=5 准确率 0.95

K=7 准确率 0.95

K=9 准确率 0.95

结论：

较适中的K值能提升模型稳定性

K值过小 (K=1) 会导致过拟合



五、过拟合分析



KNN过拟合原因

- 1. K值过小 ($K=1$)
 - 模型过于依赖局部样本, "谁离我最近就跟谁"
 - 对噪声点和异常值非常敏感
 - 训练集效果好但测试集效果下降
- 2. 特征维度过高 (维度灾难)
 - 高维空间中样本分布极其稀疏
 - 最近邻失去物理意义
- 3. 训练数据量太少或含噪声
 - 小训练集+错误标签→无法通过多数投票纠错
- 4. 距离度量未标准化
 - 特征尺度差异大+小K值→对局部波动反应过度

过拟合总结与解决方案

- KNN中过拟合的主要原因：
 - 1. K值过小 ($K=1$)
 - → 对噪声敏感，记住个别样本
 - 2. 特征维度过高
 - → 维度灾难，最近邻失去意义
 - 3. 训练数据量太少/含噪声
 - → 无法通过多数投票纠错
 - 4. 距离度量未标准化
 - → 过分依赖某一维度的微小波动
- 解决方案：
 - • 选择适中的K值 (如3~9)
 - • 特征选择/降维
 - • 增加训练数据量
 - • 标准化特征尺度



➤ 六、可视化（加入性别变量） ◀

可视化修改 — 加入性别变量

实验目的：修改可视化图，将性别变量纳入考虑。

方案：

在二维可视化中，将性别变量融入展示：

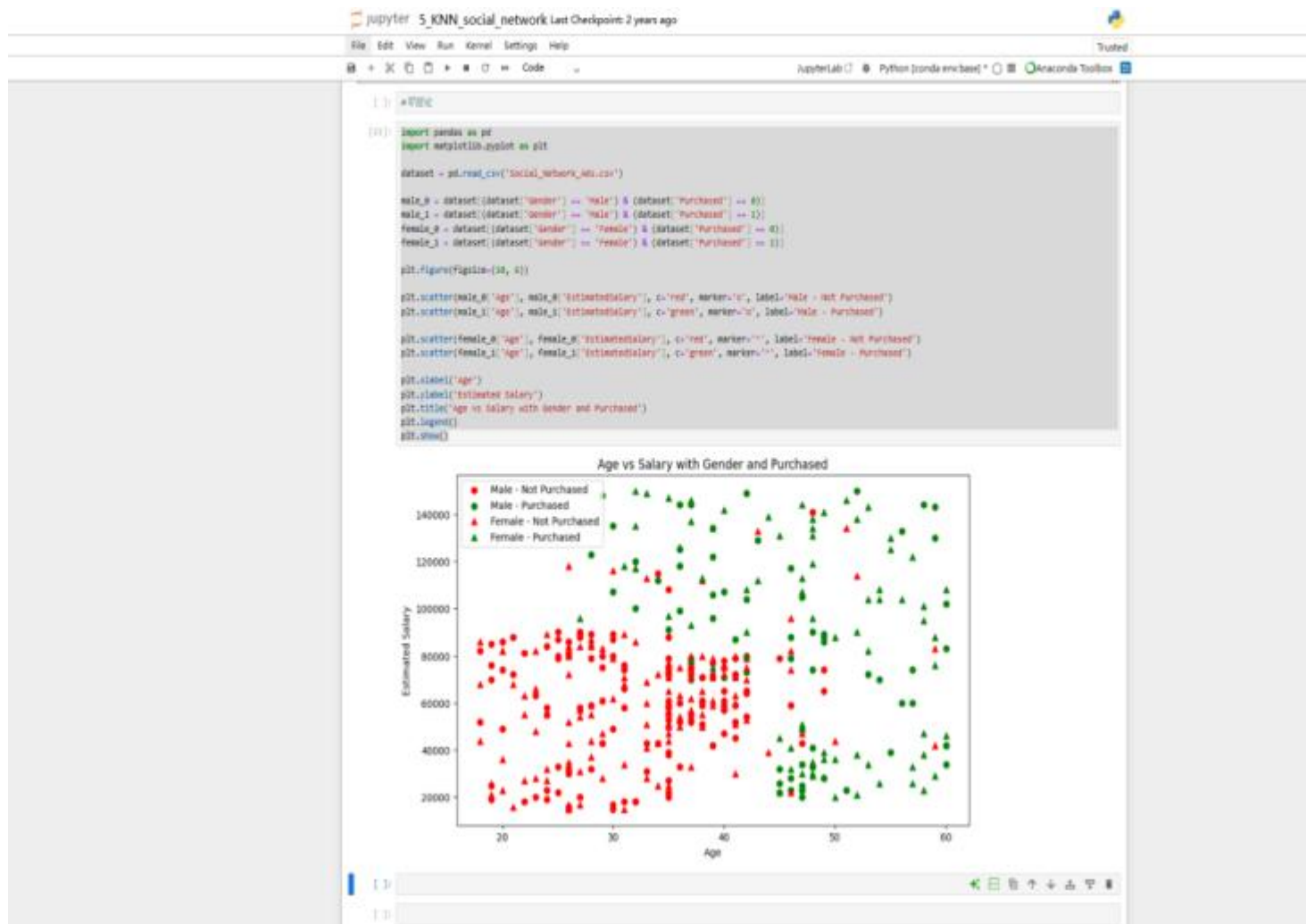
- 通过颜色区分是否购买
- 通过点的形状区分性别

效果：

使图像能够同时呈现四类信息：

1. 年龄 (x轴)
2. 收入 (y轴)
3. 购买情况 (颜色)
4. 性别 (形状)

更全面地展示数据分布特征。





THANKS FOR WATCHING



信管2302 钟京序
202321054056